**Министерство науки и высшего образования Российской Федерации**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

**«Национальный исследовательский университет ИТМО»**
**(Университет ИТМО)**

**Факультет прикладной информатики**

**Образовательная программа** Мобильные и сетевые технологии

**Направление подготовки** 09.03.03 Мобильные и сетевые технологии

# О Т Ч Е Т

## Лабораторная работа № 6.

**Тема работы:** «Работа с БД в СУБД MongoDB»

**Обучающийся:** Макаров Егор Александрович, K3240

**Преподаватель:** Говорова М.М.

Санкт-Петербург,
2025

## Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

## Программное обеспечение

Программное обеспечение: СУБД MongoDB 4+, 8.0.4 (последняя).

## Выполнение работы

### Практическое задание 2.1.1:

1) Создайте базу данных *learn*.
2) Заполните коллекцию единорогов *unicorns*.

```
test> use learn
switched to db learn
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
... db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
... db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
... db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
... db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
... db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
... db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,  gender: 'm', vampires: 39});
... db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
... db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
... db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
... db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
...
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('684332a96b141b5c957e3ce8') }
}
learn>
```

3) Используя второй способ, вставьте в коллекцию единорогов документ.

```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
...
{
  acknowledged: true,
  insertedId: ObjectId('684333c76b141b5c957e3ce9')
}
learn>
```

4) Проверьте содержимое коллекции с помощью метода *find*.



```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('684332a96b141b5c957e3cde'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('684332a96b141b5c957e3cdf'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce0'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce1'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce2'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
```

**Практическое задание 2.2.1:**

1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

   1.1)Список самок, первые 3, отсортированные по имени

```
[learn> db.unicorns.find({ gender: 'f' }).sort({ name: 1 }).limit(3);
[
  {
    _id: ObjectId('684d9a29782f9c4809bd7e71'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('684d9a5e782f9c4809bd7e75'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('684d9a90782f9c4809bd7e78'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

2.2) Список самцов, отсортированный по имени

```
learn> db.unicorns.find({ gender: 'm' }).sort({ name: 1 });
[
  {
    _id: ObjectId('684d9b74782f9c4809bd7e7b'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('684d9a14782f9c4809bd7e70'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('684d99c304536d64a7c6504a'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('684d9a6c782f9c4809bd7e76'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('684d9a99782f9c4809bd7e79'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('684d9a76782f9c4809bd7e77'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('684d9a41782f9c4809bd7e73'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
```

2) Найдите всех самок, которые любят *carrot*.
   Ограничьте этот список первой особью с помощью функций *findOne* и *limit*.

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('684332a96b141b5c957e3cdf'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('684332a96b141b5c957e3cdf'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

**Практическое задание 2.2.2:**

1) Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('684332a96b141b5c957e3cde'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce0'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce1'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce4'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce5'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('684332a96b141b5c957e3ce7'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('684333c76b141b5c957e3ce9'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
learn>
```

**Практическое задание 2.2.3:**

1) Вывести список единорогов в обратном порядке добавления.

```
learn> db.unicorns.find({gender:'m'},{loves:0,gender:0}).sort({name:1});
[
  {
    _id: ObjectId('684d9b74782f9c4809bd7e7b'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('684d9a14782f9c4809bd7e70'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('684d99c304536d64a7c6504a'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('684d9a6c782f9c4809bd7e76'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('684d9a99782f9c4809bd7e79'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('684d9a76782f9c4809bd7e77'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('684d9a41782f9c4809bd7e73'),
    name: 'Roooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('684d9a37782f9c4809bd7e72'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
```

**Практическое задание 2.2.4:**

1) Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
learn> db.unicorns.find({}, {name: 1, _id: 0, loves: { $slice: 1 }});
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

**Практическое задание 2.3.1:**

1) Вывести список самок единорогов весом от 500 до 700 кг, исключив вывод идентификатора.

```
learn> db.unicorns.find(
...   {gender: 'f', weight: {$gte: 500, $lte: 700}},
...   {_id: 0}
... )
[...
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

**Практическое задание 2.3.2:**

1) Вывести список самцов единорогов весом от 500кг и предпочитающих *grape* и *lemon*, исключив вывод идентификатора.

```
learn> db.unicorns.find(
...   {
...     gender: 'm',
...     weight: {$gte: 500},
...     loves: {$all: ['grape', 'lemon']}
...   },
...   {_id: 0}
... )
[...
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

**Практическое задание 2.3.3:**

1) Найти всех единорогов, не имеющих ключ *vampires*.

```
learn> db.unicorns.find({vampires: {$exists: false}})
[...
[
  {
    _id: ObjectId('684332a96b141b5c957e3ce8'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

**Практическое задание 2.3.4:**

1) Вывести список упорядоченный имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
...   {gender: 'm'},
...   {name: 1, loves: {$slice: 1}, _id: 0}
... ).sort({name: 1})
[...
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

**Практическое задание 3.1.1:**

1) Создайте коллекцию *towns*, включающую следующие документы:

```
learn> db.towns.insertMany([
...   {
...     name: "Punxsutawney",
...     populatiuon: 6200,
...     last_sensus: ISODate("2008-01-31"),
...     famous_for: [""],
...     mayor: { name: "Jim Wehrle" }
...   },
...   {
...     name: "New York",
...     populatiuon: 22200000,
...     last_sensus: ISODate("2009-07-31"),
...     famous_for: ["status of liberty", "food"],
...     mayor: { name: "Michael Bloomberg", party: "I" }
...   },
...   {
...     name: "Portland",
...     populatiuon: 528000,
...     last_sensus: ISODate("2009-07-20"),
...     famous_for: ["beer", "food"],
...     mayor: { name: "Sam Adams", party: "D" }
...   }
... ])
[...
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68433b5c6b141b5c957e3cea'),
    '1': ObjectId('68433b5c6b141b5c957e3ceb'),
    '2': ObjectId('68433b5c6b141b5c957e3cec')
  }
}
learn>
```

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (*party="I"*). Вывести только название города и информацию о мэре.

```
learn> db.towns.find({ "mayor.party": "I" }, { name: 1, mayor: 1, _id: 0 });
[
  {
    name: 'New York',
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
```

3) Сформировать запрос, который возвращает список городов с независимыми мэрами (*party="I"*). Вывести только название города и информацию о мэре.

```
 ]
learn> db.towns.find({ "mayor.party": { $exists: false } }, { name: 1, mayor: 1, _id: 0 });
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn>
```

**Практическое задание 3.1.2:**

1) Сформировать функцию для вывода списка самцов единорогов.
2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.
3) Вывести результат, используя *forEach*.
4) Содержание коллекции единорогов *unicorns*:

```
learn> function printMales(){var cursor=db.unicorns.find({gender:'m'}).sort({name:1}).limit(2);cursor.forEach(function(u){printjson(u)})};printMales();
{
  _id: ObjectId('684d9b74782f9c4809bd7e7b'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('684d99c304536d64a7c6504a'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
```

**Практическое задание 3.2.1:**

1) Вывести количество самок единорогов весом от 500 до 600 кг.

```
learn> db.unicorns.find({
...    gender: 'f',
...    weight: {$gte: 500, $lte: 600}
... }).count()
[...
2
learn>
```

**Практическое задание 3.2.2:**

1) Вывести список предпочтений.

```
learn> db.unicorns.distinct("loves")
[...
[
  'apple',      'carrot',
  'chocolate',  'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

**Практическое задание 3.2.3:**

1) Посчитать количество особей единорогов обоих полов.

```
learn> db.unicorns.aggregate([{$group: { _id: "$gender", count: { $sum: 1 }}}])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn>
```

**Практическое задание 3.3.1:**

1) Выполнить команду:

``` *db.unicorns.save({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})* ```2) Проверить содержимое коллекции *unicorns*.
```

```
learn> db.unicorns.insertOne({
...   name: 'Barny',
...   loves: ['grape'],
...   weight: 340,
...   gender: 'm'
... })
[...
{
  acknowledged: true,
  insertedId: ObjectId('68433ebd6b141b5c957e3ced')
}
learn> db.unicorns.find({name: 'Barny'})
[...
[
  {
    _id: ObjectId('68433ebd6b141b5c957e3ced'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn>
```

**Практическое задание 3.3.2:**

1) Для самки единорога *Ayna* внести изменения в БД: теперь ее вес 800, она убила 51 вапмира.

```
learn> db.unicorns.updateOne(
...   {name: "Ayna"},
...   {
...     $set: {
...       weight: 800,
...       vampires: 51
...     }
...   }
... )
[...
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>
```

2) Проверить содержимое коллекции *unicorns*.

```
{
  _id: ObjectId('684d9a5e782f9c4809bd7e75'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 800,
  gender: 'f',
  vampires: 51
```

**Практическое задание 3.3.3:**

1) Для самца единорога *Raleigh* внести изменения в БД: теперь он любит рэдбул.

2) Проверить содержимое коллекции *unicorns*.

```
},
{
  _id: ObjectId('684d9a76782f9c4809bd7e77'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar', 'redbull' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
```

**Практическое задание 3.3.4:**

1) Всем самцам единорогов увеличить количество убитых вапмиров на 5.

2) Проверить содержимое коллекции *unicorns*.

```
learn> db.unicorns.updateMany({gender:'m'},{$inc:{vampires:5}});db.unicorns.find().pretty();
[
  {
    _id: ObjectId('684d99c304536d64a7c6504a'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('684d9a14782f9c4809bd7e70'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('684d9a29782f9c4809bd7e71'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('684d9a37782f9c4809bd7e72'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('684d9a41782f9c4809bd7e73'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('684d9a4c782f9c4809bd7e74'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('684d9a5e782f9c4809bd7e75'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  },
  {
```

**Практическое задание 3.3.5:**

1) Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.

2) Проверить содержимое коллекции *towns*.

```
{
  _id: ObjectId('684da65c3e0db7e83d88148e'),
  name: 'Portland',
  populatiuon: 528000,
  last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
  famous_for: [ 'beer', 'food' ],
  mayor: { name: 'Sam Adams' }
}
```

**Практическое задание 3.3.6:**

1) Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.

2) Проверить содержимое коллекции unicorns.

```
{
  _id: ObjectId('684d9a99782f9c4809bd7e79'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon', 'chocolate' ],
  weight: 650,
  gender: 'm',
  vampires: 59
}
```

**Практическое задание 3.3.7:**

1. *Изменить информацию о самке единорога* `Aurora`: *теперь она любит еще и сахар, и лимоны.*

2. *Проверить содержимое коллекции* `unicorns`.

```
},
{
  _id: ObjectId('684d9a29782f9c4809bd7e71'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
```

**Практическое задание 3.4.1:**

1) Создайте коллекцию *towns*, включающую следующие документы:

```
learn> db.towns.insert({ name: "Punxsutawney", popujatiuon: 6200, last_sensus: ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: { name: "Jim Wehrle" } });
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('684dac0dc15c621bdbee4875') }
}
learn> db.towns.insert({ name: "New York",     popujatiuon: 22200000, last_sensus: ISODate("2009-07-31"), famous_for: ["status of liberty","food"], mayor: { name: "Michael Bloomberg", party: "I" } });
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('684dac1dc15c621bdbee4876') }
}
learn> db.towns.insert({ name: "Portland",     popujatiuon: 528000,   last_sensus: ISODate("2009-07-20"), famous_for: ["beer","food"], mayor: { name: "Sam Adams",        party: "D" } });
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('684dac31c15c621bdbee4877') }
}
learn>
```

2) Удалите документы с беспартийными мэрами.

3) Проверьте содержание коллекции.

```
}
learn> db.towns.deleteMany({ "mayor.party": { $exists: false } }); db.towns.find().pretty();
[
  {
    _id: ObjectId('684da6393e0db7e83d88148d'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('684dac1dc15c621bdbee4876'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('684dac31c15c621bdbee4877'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
```

4) Очистите коллекцию.

```
]
learn> db.towns.deleteMany({});
{ acknowledged: true, deletedCount: 3 }
```

5) Просмотрите список доступных коллекций.

```
{ acknowledged: true, deletedCount: 3 }
learn> show collections;
towns
unicorns
users
```

**Практическое задание 4.1.1:**

1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

   db.habitats.insert({ _id: 'forest', name: 'Enchanted Forest', description: 'Зачарованный лес, полный магии' });
   db.habitats.insert({ _id: 'mountains', name: 'Misty Mountains', description: 'Туманные горы с скрытыми долинами' });
   db.habitats.insert({ _id: 'meadow', name: 'Sunny Meadow', description: 'Солнечный луг, усыпанный цветами' });

2) Включите для нескольких единорогов в документы ссылку на зону обитания, использую второй способ автоматического связывания.

```
[learn> db.unicorns.update({ name: 'Horny' },  { $set: { habitat: { $ref: 'habitats', $id: 'forest' } } });
DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.update({ name: 'Aurora' }, { $set: { habitat: { $ref: 'habitats', $id: 'meadow' } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.update({ name: 'Pilot' },  { $set: { habitat: { $ref: 'habitats', $id: 'mountains' } } });
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

3) Проверьте содержание коллекции единорогов.

```
learn> db.unicorns.find().pretty();
[
  {
    _id: ObjectId('684d99c304536d64a7c6504a'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    habitat: DBRef('habitats', 'forest')
  },
  {
    _id: ObjectId('684d9a14782f9c4809bd7e70'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('684d9a29782f9c4809bd7e71'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43,
    habitat: DBRef('habitats', 'meadow')
  },
  {
    _id: ObjectId('684d9a37782f9c4809bd7e72'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('684d9a41782f9c4809bd7e73'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('684d9a4c782f9c4809bd7e74'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('684d9a5e782f9c4809bd7e75'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
```

**Практическое задание 4.2.1:**

1) Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа name с флагом *unique*.

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true }); db.unicorns.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

**Практическое задание 4.3.1:**

1) Получите информацию о всех индексах коллекции *unicorns*.

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true }); db.unicorns.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

2) Удалите все индексы, кроме индекса для идентификатора.

```
learn> db.unicorns.dropIndexes();
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

3) Попытайтесь удалить индекс для идентификатора.

```
learn> db.unicorns.dropIndex("_id_");
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

**Практическое задание 4.4.1:**

1) Создайте объемную коллекцию numbers, задействовав курсор: ```*for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}*```

```
learn> for (i = 0; i < 100000; i++) db.numbers.insert({ value: i });
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('684db1b46f8536d4019a690f') }
}
```

2) Выберите последних четыре документа.

```
learn> db.numbers.find().sort({_id:-1}).limit(4);
[
  { _id: ObjectId('684db1b46f8536d4019a690f'), value: 99999 },
  { _id: ObjectId('684db1b46f8536d4019a690e'), value: 99998 },
  { _id: ObjectId('684db1b46f8536d4019a690d'), value: 99997 },
  { _id: ObjectId('684db1b46f8536d4019a690c'), value: 99996 }
]
```

3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра executionTimeMillis)

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 2,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
```

4) Создайте индекс для ключа value.

```
}
learn> db.numbers.createIndex({value:1});
value_1
learn>
```

5) Получите информацию о всех индексах коллекции numbres.

```
value_1
learn> db.numbers.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

6) Выполните запрос 2.

```
]
learn> db.numbers.find().sort({_id:-1}).limit(4);
[
  { _id: ObjectId('684db1b46f8536d4019a690f'), value: 99999 },
  { _id: ObjectId('684db1b46f8536d4019a690e'), value: 99998 },
  { _id: ObjectId('684db1b46f8536d4019a690d'), value: 99997 },
  { _id: ObjectId('684db1b46f8536d4019a690c'), value: 99996 }
]
learn>
```

7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
```

8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

**Ответ:** Запрос с использованием индекса выполняется значительно быстрее (около 0 мс) по сравнению с запросом без индекса (2 мс), так как индекс позволяет избежать полного просмотра коллекции. Запрос с использованием индекса намного эффективнее.

## Контрольные вопросы:

### Пункт 3

1. **Для чего служит «точка» (`.`)?**
   Символ «точка» позволяет обратиться к вложенным полям документа или к элементам массива. Например, `user.address.city` указывает на поле `city` внутри объекта `address`, а `doc.tags.0` – на первый элемент массива `tags`.
2. **Как задействовать курсор?**
   Курсор — это объект, возвращаемый `find()`, который даёт доступ к результатам пошагово. С его помощью можно:
   - Перебирать результаты по одному документу (`cursor.next()`, `cursor.hasNext()`).
   - Ограничивать и смещать выдачу (`cursor.limit()`, `cursor.skip()`).
   - Сортировать данные (`cursor.sort()`).
   - Применять JavaScript-функции к каждому элементу (`cursor.forEach()`, `cursor.map()`).
3. **Что умеет Aggregation Framework?**
   Конвейер агрегации даёт набор «ступеней» и операторов для гибкой обработки данных:
   - **Этапы** (`$match`, `$group`, `$project`, `$sort`, `$lookup` и т. д.) формируют последовательность преобразований.
   - **Агрегационные операторы** (`$sum`, `$avg`, `$max`, `$min`, `$push` и прочие) подсчитывают, усредняют, собирают списки.
   - **Встроенные вычисления** позволяют делать арифметику, работу со строками и датами прямо внутри конвейера.
   - **`$lookup`** выполняет связывание (join) между коллекциями.
4. **Что гибче: `save()` или `update()`?**
   Метод `updateOne`/`updateMany` предоставляет более тонкую настройку:
   - Можно выборочно менять поля через `$set`, `$unset`, `$inc` и другие операторы.
   - Задавать условия для выборки документов.
   - Добавлять или удалять элементы в массивах (`$addToSet`, `$pull`).
     В отличие от них, `save()` просто заменяет весь документ с данным `_id` или вставляет новый, не позволяя частично модифицировать только нужные поля.
5. **Как удаляются документы по умолчанию?**
   Удаляющие методы `deleteOne()` и `deleteMany()` физически убирают записи из коллекции без какого-либо «мягкого» удаления или архивирования.

### Пункт 4

1. **Какие есть способы связать коллекции?**
   - **Вложение (Embedding):** хранить всё прямо внутри одного документа.
   - **Ссылки (References):** сохранять `_id` других документов и при необходимости подтягивать их через `$lookup`.
   - **Денормализация:** дублировать нужные поля из связанной коллекции прямо в документ, чтобы избежать джойнов.
2. **Сколько можно создать индексов в одной коллекции?**
   Технического жёсткого потолка нет, но на практике не стоит превышать ~64 индекса, иначе это ударит по производительности вставок и обновлений.
3. **Как посмотреть все индексы коллекции?**
   Вызвать метод

   ```
   db.<имя_коллекции>.getIndexes()
   ```

   — это вернёт список объектов с ключами и свойствами каждого индекса.

**Вывод:**
В ходе работы я познакомился с базовыми операциями MongoDB: созданием, чтением, обновлением и удалением документов (CRUD). Освоил применение оператора «точка» для доступа к вложенным данным и приёмов работы с курсором для управления большими объёмами. Изучил агрегирование через конвейеры и научилась проектировать индексы для ускорения запросов. Всё это позволяет гибко моделировать данные, выбирать схемы (вложение vs. ссылки) и оптимизировать производительность NoSQL-решения под реальные задачи.